

DevOps Technical Challenge: Sentinel Split Architecture

Time Limit: 3 Days.

Please reach out to Omrip@rapyd.net for your credentials for the AWS account of this challenge. Once the credentials are received you will have 72 hours to submit before access is closed. Good luck!

Please read carefully:

For this challenge you are intentionally not given full AWS permissions. The provided access is deliberately scoped and is part of the exercise. The challenge is designed to be solvable within these constraints and has been successfully completed by multiple engineers. We are evaluating your ability to design secure infrastructure under realistic guardrails, with a strong emphasis on least privilege, clean and modular Terraform, and clear reasoning.

For EKS provisioning, you are expected to create and use IAM roles that follow the approved naming prefixes:

- `eks-` for EKS cluster and node roles
- `sentinel-` for Sentinel related roles

Only IAM roles matching these prefixes can be created, modified, or passed. You should not create or use IAM roles outside of this scope. If you encounter a permission limitation, do not attempt to bypass it. Clearly document your assumptions, trade-offs, and how you would address the limitation in a real production environment.

CI/CD requirement (mandatory):

You must implement and run this challenge via **GitHub Actions**. Do not apply Terraform manually from your local machine. Your Terraform plan/apply workflow must be executed through a GitHub Actions pipeline.

Terraform structure requirement (mandatory):

You must use **Terraform modules** (not a single flat configuration). Your solution should be modular, reusable, and clearly structured (for example: networking/IAM/EKS as separate modules or similar), with clean inputs/outputs and minimal duplication.

Background

You're joining the team working on an imaginary product called Rapyd **Sentinel**, our flagship

threat intelligence platform.

To support better scalability, compliance, and team autonomy, Sentinel's architecture has recently been split into two isolated domains:

1. **Gateway Layer (Public)** – hosts internet-facing APIs and proxies
2. **Backend Layer (Private)** – runs internal processing and sensitive services

These two environments:

- Operate in **separate AWS VPCs**
- Each host their own **Kubernetes cluster**
- Must **communicate privately and securely** across networks

You've been tasked with building a **proof-of-concept environment** that mirrors this architecture — using Terraform, EKS, and GitHub Actions CI/CD and container registry — and demonstrates production-readiness, modularity, and security best practices.

Objectives

You will:

- Use Terraform to build two isolated VPCs
- Deploy two EKS clusters (one per VPC)
- Set up secure private networking between them
- Deploy a basic app in the backend and a proxy in the gateway
- Wire everything up with a CI/CD pipeline
- Document your design, security decisions, and tradeoffs

Requirements

Infrastructure (Terraform)

- Create two AWS VPCs:
 - `vpc-gateway`: for the proxy and public-facing services
 - `vpc-backend`: for the internal backend services
 - Each VPC must include:
 - Two **private subnets** in different Availability Zones
 - NAT Gateways (for outbound traffic if needed)
 - No public EC2s or unrestricted access
 - Create **VPC Peering** (or optionally **Transit Gateway**) between the VPCs
 - Set up correct **routing tables** and **security groups** to allow cross-VPC private communication
 - Deploy one **Amazon EKS cluster per VPC**:
 - `eks-gateway` in `vpc-gateway`
 - `eks-backend` in `vpc-backend`
 - Use **Terraform modules** for reusable, clean structure
-

Application Workloads (Kubernetes)

- In `eks-backend`:
 - Deploy a **simple internal backend service** (e.g., a basic web server that responds "Hello from backend")
 - **Do not expose it to the internet**
- In `eks-gateway`:
 - Deploy a **proxy application** (e.g., NGINX reverse proxy or a simple Node.js forwarder)
 - The proxy **exposes a public LoadBalancer**

- All traffic to the proxy is forwarded to the backend service over the VPC link
- Configure DNS resolution or hardcoded IPs as needed
- Validate that the **proxy successfully reaches the backend**
- Restrict network access to the backend service using **Security Groups**, allowing only traffic from the EKS nodes (or CIDR) in the proxy cluster's VPC.
- Optionally, apply **Kubernetes NetworkPolicy** to further limit internal pod communication within the backend cluster.

CI/CD Pipeline (GitHub Actions)

- Set up a GitHub Actions workflow to:
 - Validate Terraform (`terraform validate`, `tflint`)
 - Plan and apply Terraform
 - Validate Kubernetes manifests (`kubeval`, `kubectl apply --dry-run`)
 - Deploy the proxy and backend services
- Workflow must be triggered on push
- **Optional Bonus:** Use GitHub **OIDC federation** to deploy to AWS without long-lived credentials

Documentation (README.md)

Include clear instructions and justifications:

- How to clone and run the project
- How networking is configured between VPCs & clusters
- How the proxy talks to the backend
- NetworkPolicy explanation and security model
- CI/CD pipeline structure
- **Trade-offs** due to the 3-day limit

- **Optional: Cost optimization notes** (e.g., NAT usage, instance types, load balancer selection)
- What you would do next (e.g., TLS/mTLS, observability, GitOps, ingress controllers, service mesh, Vault, etc.)

Evaluation Criteria

We'll be looking for:

- Infrastructure correctness and security (private subnets, no public EC2s, tight SGs)
- Terraform quality: modular, readable, maintainable
- Kubernetes setup and working cross-cluster communication
- CI/CD automation, linting, and deploy flow
- Networking clarity and security (VPC peering, SGs, NetworkPolicy)
- Realism in design trade-offs and next steps
- Documentation completeness and clarity